Парсер вакансий Habr Career: требования и архитектура

Oct 14, 2026 · @Eiden

Цель и границы

Проект даёт ежедневно обновляемый датасет вакансий Habr Career и аналитику рынка junior-позиций в данных, аналитике и ML. Это одновременно портфолио и инструмент для твоего собственного поиска работы.

Входит в проект:

- ежедневный сбор вакансий выбранных категорий (аналитика, data science, ML, CV, data engineering);
- хранение с историей изменений, а не только последний срез;
- аналитические витрины и дашборд;
- одна-две ML-задачи на текстах вакансий;
- Telegram-бот с уведомлениями о новых вакансиях.

Не входит (пока): другие площадки, публичный API, сбор данных о кандидатах, коммерческое использование данных.

Критерий успеха: репозиторий запускается одной командой, сбор идёт без твоего участия минимум 60 дней подряд, а по накопленным данным опубликован разбор с честно описанными ограничениями выборки.

Функциональные требования

Каждое требование проверяемо: можно ответить «сделано / не сделано». Приоритет: **P0** — без этого проекта нет, **P1** — нужно для сильного портфолио, **P2** — по желанию.

№	Требование	Приоритет
F1	Сбор списка вакансий по заданным категориям и страницам, с пагинацией	P0

№	Требование	Приоритет
F2	Загрузка карточки каждой вакансии; сырой ответ сохраняется на диск до любой обработки	P0
F3	Парсинг в поля: название, компания, зарплата (от/до/валюта), грейд, формат работы, город, навыки, описание, дата публикации, URL	P0
F4	Идемпотентность: повторный запуск за тот же день не создаёт дублей	P0
F5	История: фиксируются первое и последнее появление вакансии, дата закрытия, изменения полей между снимками	P1
F6	Нормализация: единая валюта, единые названия навыков, определение грейда и удалёнки	P1
F7	Перепарсинг: любой старый сырой снимок можно заново разобрать новой версией парсера	P1
F8	Проверки качества данных после каждого запуска (доля пустых полей, резкое падение числа вакансий, выбросы зарплат) с отчётом	P1
F9	SQL-витрины и дашборд: спрос на навыки по грейдам, медианы зарплат, доля удалёнки, срок жизни вакансий, динамика по неделям	P1
F10	Извлечение навыков из текста описания с оценкой качества на ручной разметке (не менее 100 вакансий)	P1
F11	Предсказание зарплаты для вакансий без вилки с честной метрикой и разбором ошибок	P2
F12	Поиск похожих вакансий по эмбеддингам описаний	P2

№	Требование	Приоритет
F13	Telegram-бот: уведомление о новых вакансиях по фильтрам (грейд, навыки, формат)	P2
F14	Планировщик: ежедневный запуск без ручного вмешательства, с логами и уведомлением об ошибке	P0

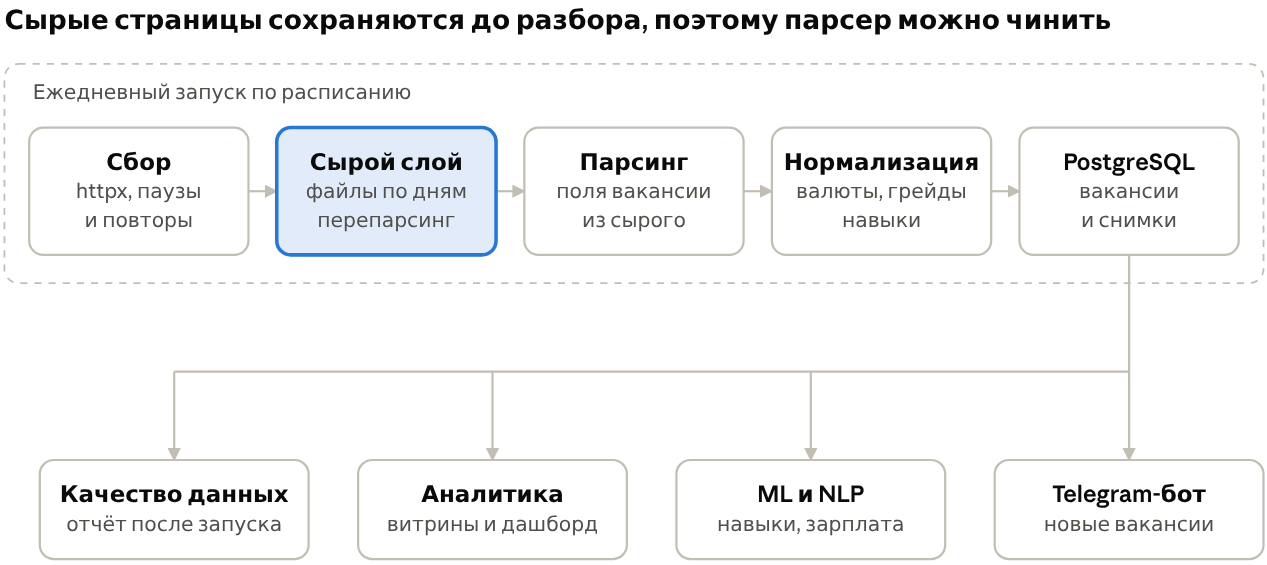
Нефункциональные требования

Этот раздел отличает учебный скрипт от проекта, который не стыдно показать на собеседовании.

- **Этика и ограничения сбора.** Перед стартом прочитай `robots.txt` и условия использования Habr Career; я их не проверял, так что решение за тобой. Держи паузу между запросами (начни с 2–3 секунд), один поток, честный User-Agent с контактом, ни одного обхода защиты. Не публикуй сырые данные целиком: публикуй агрегаты и выводы.
- **Надёжность.** Сбой одной вакансии не роняет весь запуск: ошибка логируется, запуск продолжается. Повторы при временных ошибках с растущей паузой.
- **Воспроизводимость.** Проект поднимается командой `docker compose up`, зависимости зафиксированы, настройки лежат в `.env` и конфиге, секреты не попадают в git.
- **Наблюдаемость.** Структурные логи; в конце каждого запуска сводка: сколько страниц, вакансий, новых, закрытых, ошибок и время работы.
- **Качество данных.** Парсер покрыт тестами на сохранённых HTML-примерах; при смене вёрстки тесты падают раньше, чем портится база.
- **Данные.** Схема версионируется миграциями, а не ручными правками в БД.
- **Качество кода.** Линтер и форматтер, типизация, CI с прогоном тестов на каждый коммит.
- **Документация.** README с целью, схемой архитектуры, командами запуска, примерами выводов и разделом «ограничения данных».

Архитектура

Архитектура — конвейер из шести слоёв: каждый читает результат предыдущего и ничего не знает о его устройстве. Сырые страницы сохраняются до разбора, поэтому ошибку парсера можно исправить и заново обработать старые данные без повторных запросов к сайту.



архитектура · 5 шагов конвейера и 4 потребителя данных

Слева направо: сбор, сырые файлы, парсинг, нормализация, база. Из базы данные читают проверки качества, аналитика, ML и бот; весь конвейер запускается раз в сутки.

Выбор технологий. Начинай с самого простого варианта в каждой строке и усложняй, только когда упрёшься в ограничение.

Слой	Инструмент	Почему так
Сбор	`httpx` + `selectolax` (или BeautifulSoup); Playwright только если страница не отдаёт данные без браузера	Быстро и легко отлаживать; браузер тяжёлый и хрупкий
Сырое хранилище	Файлы на диске: `data/raw/ДАТА/`	Дёшево, позволяет перепарсинг

Слой	Инструмент	Почему так
База данных	PostgreSQL (на старте допустимы SQLite или DuckDB)	Реальный рабочий стек и практика SQL
Миграции	Alembic	Схема меняется под контролем
Обработка	SQL + pandas или polars	SQL ты будешь показывать на собеседованиях
Планировщик	cron или GitHub Actions по расписанию; позже Prefect	Начни с простого, оркестратор нужен не сразу
Дашборд	Streamlit или Metabase	Streamlit — Python и быстрый результат; Metabase — типичный BI-опыт
ML	scikit-learn для базовых решений; эмбеддинги при необходимости	Сначала простая модель как точка отсчёта, потом сложнее
Бот	aiogram	Популярная библиотека для Telegram
Упаковка и CI	Docker Compose, GitHub Actions	Воспроизводимый запуск и автоматические тесты

Если постоянно включённого компьютера или сервера нет, сбор можно запускать по расписанию через GitHub Actions, а данные хранить во внешней БД с бесплатным тарифом.

Модель данных

Главная идея: вакансия хранится как сущность плюс ряд ежедневных снимков. Так видно, когда она появилась, как менялась и когда закрылась. Поля ниже — стартовый минимум, расширяй по мере необходимости.

Таблица	Ключевые поля	Назначение
`raw_pages`	id, url, fetched_at, http_status, path_to_file, content_hash	Учёт сохранённых сырых страниц; основа для перепарсинга
`vacancies`	vacancy_id (из URL), title, company_id, grade, employment_type, remote_flag, city, salary_from, salary_to, currency, description, url, first_seen_at, last_seen_at, closed_at	Текущее состояние вакансии
`vacancy_snapshots`	vacancy_id, snapshot_date, content_hash, salary_from, salary_to, title	Ежедневный срез; по смене hash видно, что изменилось
`companies`	company_id, name, url	Справочник работодателей
`skills`	skill_id, canonical_name, aliases	Словарь навыков с синонимами («postgres» = «PostgreSQL»)
`vacancy_skills`	vacancy_id, skill_id, source (`habr_tags` / `text_rules` / `model`)	Связь вакансий и навыков; source позволяет сравнивать методы
`pipeline_runs`	run_id, started_at, finished_at, pages, new, closed, errors	История запусков и контроль качества

Правила:

- Вакансия считается закрытой, если её не было в выдаче N дней подряд (начни с 3), а не после первого же пропуска.
- Зарплата хранится в исходной валюте и отдельно в рублях с датой курса; вилки без указания "от" или "до" не придумываются.
- Пустое значение остаётся пустым (`NULL`), его нельзя заполнять нулём или средним.

Структура репозитория и стандарты кода

Структура разделяет слои так же, как архитектура: каждый можно тестировать и менять отдельно.

```
habr-vacancies/
├── README.md
├── docker-compose.yml
├── pyproject.toml
├── .env.example
├── src/
│   └── vacancies/
│       ├── collect/      # HTTP-клиент, пагинация, сохранение сырого
│       ├── parse/        # сырое -> структурированные поля
│       ├── normalize/    # валюты, грейды, навыки, дедупликация
│       ├── storage/      # подключение к БД, миграции, репозитории
│       ├── quality/      # проверки качества данных
│       ├── analytics/    # SQL-витрины и расчёты
│       ├── ml/           # извлечение навыков, зарплата, эмбеддинги
│       ├── bot/          # Telegram-бот
│       └── pipeline.py   # точка входа ежедневного запуска
├── dashboard/            # приложение дашборда
├── tests/
│   ├── fixtures/         # сохранённые HTML-примеры
│   └── ...
├── notebooks/            # исследования, не продакшен-код
└── docs/                 # архитектура, ограничения данных, выводы
```

Стандарты:

- Python 3.11+, зависимости через `uv` или `poetry`, версии зафиксированы.
- `ruff` (линтер и форматтер) и `mypy` или `pyright` для типов; запускаются в CI.
- `pytest`; парсер тестируется на файлах из `tests/fixtures`, без обращения к сети.
- Git: небольшие коммиты с понятными сообщениями, ветка на задачу, main всегда рабочая.
- Парсинг, сеть и работа с БД не смешиваются в одной функции.

Этапы и критерии готовности

Сроки — ориентир для человека с базой, но без большого опыта: если этап идёт дольше, это нормально. Переходи к следующему только когда выполнен критерий готовности. **Сбор данных запускай как можно раньше** (этап 3), чтобы история копилась, пока ты занят остальным.

Этап	Что делаем	Критерий готовности	Ориентир
0. Разведка	Читаешь `robots.txt` и условия сайта, смотришь, как отдаются данные (HTML или JSON во вкладке Network), выбираешь категории вакансий, создаёшь репозиторий	Есть короткая заметка в `docs/`: откуда берём данные, какие поля доступны, какой темп запросов	2–3 дня
1. Сбор	HTTP-клиент с паузами и повторами, обход списка и карточек, сохранение сырого на диск	Скрипт выгружает все вакансии выбранных категорий за один запуск; повторный запуск не делает лишних запросов	1 неделя
2. Парсинг и хранилище	Разбор полей, схема БД с миграциями, запись вакансий и снимков, тесты на фикстурах	Данные лежат в БД; тесты проходят без сети; перепарсинг сырых файлов работает	1–2 недели
3. Автоматизация	Docker Compose, ежедневный запуск по расписанию, логи, сводка запуска, уведомление об ошибке	Сбор идёт 7 дней подряд без твоего участия	1 неделя

Этап	Что делаем	Критерий готовности	Ориентир
4. Качество и нормализация	Проверки качества, справочник навыков, валюты, грейды, дедупликация	После каждого запуска есть отчёт качества; доля пустых ключевых полей известна и объяснена	1–2 недели
5. Аналитика	SQL-витрины, дашборд, ответы на 8–10 вопросов к данным	Дашборд открывается по ссылке; для каждого вывода указано, на каких данных он построен	2 недели
6. ML и NLP	Извлечение навыков с оценкой на ручной разметке; затем одна из задач F11 или F12	Есть метрики на отложенной выборке, сравнение с простым базовым решением, разбор ошибок	3–4 недели
7. Бот и оформление	Telegram-бот, README, схема архитектуры, CI, публикация разбора	Незнакомый человек запускает проект по README с первого раза; разбор опубликован	1–2 недели

Параллельно с этапом 3 и дальше накапливается история: к этапу 7 у тебя должно быть минимум 6–8 недель ежедневных данных, а к публикации разбора — около 3 месяцев.

Риски и Definition of Done

Риск	Чем грозит	Как снижаем
Сайт меняет вёрстку	Парсер ломается, в данных появляются пропуски	Сырое хранится всегда; тесты на фикстурах; проверка доли пустых полей после запуска; перепарсинг

Риск	Чем грозит	Как снижаем
Сайт ограничивает или блокирует запросы	Сбор останавливается	Низкий темп, один поток, честный User-Agent; при блокировке не обходить защиту, а замедлиться или остановиться
Правила сайта запрещают сбор	Проект нельзя публиковать как задумано	Проверить на этапе 0; при запрете выбрать другой источник или работать только с тем, что явно разрешено
Мало данных для ML	Метрики нестабильны	Не обещать точности; показывать доверительные интервалы и сравнение с простым базовым решением
Проект растёт вширь и не заканчивается	Нет готового результата	Не переходить к P2-требованиям, пока не закрыты все P0 и P1; фиксировать объём каждого этапа заранее
Мотивация падает без внешнего дедлайна	Проект остаётся на 60%	Один маленький, но завершённый шаг в день; публичный репозиторий с регулярными коммитами

Definition of Done — проект готов, когда выполнено всё:

- [ ] Репозиторий публичный, README объясняет цель, архитектуру и запуск
- [ ] `docker compose up` поднимает проект на чистой машине
- [ ] Сбор идёт автоматически минимум 60 дней подряд, пропуски задокументированы
- [ ] Схема БД под миграциями, есть история вакансий
- [ ] Тесты проходят в CI, парсер покрыт фикстурами
- [ ] После каждого запуска формируется отчёт качества данных

- [ ] Дашборд доступен по ссылке
- [ ] Извлечение навыков оценено на ручной разметке, метрики записаны в README
- [ ] Опубликован разбор рынка с разделом «ограничения данных»
- [ ] Ты можешь за 5 минут объяснить на собеседовании любое решение в архитектуре и почему оно такое